

Adversarial Camouflage para Deteccion Aerea con YOLOv8-OBB

Optimizacion Diferenciable de Texturas para Evasion de Detecciones en
Imágenes DOTA

Christian Echeverria (221441)

Diederich Solis (22952)

Universidad del Valle de Guatemala

CC3045 — Inteligencia Artificial

27 de mayo de 2026

Resumen

Este proyecto estudia la vulnerabilidad de los detectores de objetos en imágenes aéreas ante texturas adversariales aplicadas sobre vehículos. Se utiliza YOLOv8-OBB como modelo víctima y el dataset DOTA v1.0 como fuente de escenas aéreas con objetos orientados. El trabajo evolucionó desde pruebas iniciales de oclusión con texturas fijas — que no constituyen un ataque adversarial — hacia un ataque adversarial real, donde la textura es un tensor optimizable mediante descenso de gradiente y se aplica a través de operaciones diferenciables de *torch* y *kornia*. Para el objeto sobre el cual se optimizó la textura, YOLOv8-OBB detectaba inicialmente tres cajas de la clase *large vehicle* con confianzas de 0.82, 0.80 y 0.79; tras aplicar la textura adversarial, el número de detecciones dentro de la misma región descendió a cero. Una evaluación de transferencia sobre 20 objetos de tres escenas distintas alcanzó una tasa de éxito del 75 % (15/20), con un comportamiento revelador: la textura elimina la detección en el 100 % de los objetos de escenas visualmente similares a la de optimización, pero falla por completo en una escena fuera de esa distribución. Estos resultados validan el *pipeline* de ataque digital, cuantifican su alcance y sus límites, y establecen una base reproducible para extensiones futuras.

Índice

1	Descripcion del Problema	3
1.1	Relevancia	3
2	Analisis	4
2.1	Modelo victima	4
2.2	Dataset	4
2.3	Estrategia adoptada	4
3	Propuesta de Solucion	5
4	Descripcion de la Solucion	5
4.1	Fase 1: Baseline con imagenes limpias	5
4.2	Fase 2: Texturas fijas (prueba de concepto descartada)	5
4.3	Fase 3: Ataque adversarial diferenciable	6
4.3.1	Forward diferenciable de YOLOv8-OBB	6
4.3.2	Composicion diferenciable de la textura	7
4.3.3	Funcion de perdida	7
4.3.4	Configuracion de optimizacion	8
4.4	Fase 4: Exploracion de textura universal	8
5	Herramientas Aplicadas	8
5.1	Herramientas vistas en clase	8
5.2	Herramientas no vistas en clase	9
5.3	Uso de herramientas de IA generativa	9
6	Resultados	10
6.1	Ataque sobre un objeto	10
6.2	Transferencia a otros objetos	11
6.3	Exploracion de textura universal	12
6.4	Discusion	13
7	Limitaciones	13
8	Trabajo Futuro	13
9	Conclusion	13

1 Descripcion del Problema

Los sistemas modernos de vigilancia aerea emplean detectores de objetos para identificar vehiculos, embarcaciones, aeronaves e infraestructura en imagenes capturadas desde drones o satelites. Modelos como YOLOv8-OBB permiten detectar objetos mediante cajas orientadas (*Oriented Bounding Boxes*), una capacidad especialmente util en escenarios aereos, donde los objetos aparecen rotados en cualquier direccion y a escalas pequenas.

Sin embargo, las redes neuronales de vision son sensibles a **perturbaciones adversariales**: modificaciones visuales, a menudo de apariencia inusual, disenadas deliberadamente para inducir un fallo en el modelo. En el caso de un detector, ese fallo puede consistir en reducir la confianza de una deteccion hasta hacerla desaparecer, o en provocar una clasificacion incorrecta.

El problema concreto que aborda este proyecto es el siguiente:

¿Es posible disenar una textura que, aplicada sobre la huella visual de un vehiculo en una imagen aerea, haga que YOLOv8-OBB deje de detectarlo, y que ese efecto se logre no por simple oclusion sino mediante una perturbacion optimizada contra el propio modelo?

La distincion entre **occlusion** y **ataque adversarial** es central en este trabajo. Cubrir un objeto con cualquier material opaco — una lona, un rectangulo gris — tambien impide su deteccion, pero eso no demuestra ninguna vulnerabilidad del modelo: es un hecho trivial. Un ataque adversarial genuino encuentra, mediante optimizacion, un patron especifico que explota la forma en que el detector procesa la imagen. Esa diferencia define el alcance del proyecto.

1.1 Relevancia

El problema tiene aplicacion real tanto en la industria como en la academia. En seguridad, comprender estas vulnerabilidades es necesario para auditar la robustez de sistemas de percepcion automatizada antes de desplegarlos. En investigacion, el *adversarial machine learning* es un area activa cuyo objetivo es entender los limites de generalizacion de los modelos de vision. El tema, ademas, no puede resolverse con una unica consulta a una herramienta generativa: requiere implementar un proceso iterativo de optimizacion, depuracion y evaluacion.

2 Analisis

2.1 Modelo victima

Se selecciono **YOLOv8n-OBB**, cargado mediante la libreria Ultralytics [2]. Es un detector ligero, con soporte para cajas orientadas y entrenado sobre las clases de DOTA, lo que lo hace adecuado para escenas aereas y para ejecutarse en la GPU T4 disponible en Google Colab.

Una observacion tecnica determino todo el diseno del ataque: la funcion `YOLO.predict()` de Ultralytics ejecuta inferencia con *Non-Maximum Suppression* (NMS) y **no propaga gradientes** hacia la imagen de entrada. Por ello no puede usarse para entrenar una textura. El ataque requiere acceder al modulo interno de PyTorch (`yolo.model`), que si devuelve un tensor crudo de predicciones con grafo de gradiente.

YOLOv8 es un detector *anchor-free* y, a diferencia de versiones anteriores, **no posee un termino de *objectness* separado**: la confianza de una deteccion es directamente la confianza de la clase. En consecuencia, la funcion de perdida del ataque debe construirse sobre las confianzas de clase.

2.2 Dataset

Se utilizo **DOTA v1.0**, un dataset de imagenes aereas con anotaciones de objetos orientados [1], con 15 clases entre las que se encuentran *small vehicle* y *large vehicle*. Estas dos clases se definieron como objetivo del ataque por ser las relevantes para un escenario de camuflaje de vehiculos en vista cenital. Las imagenes, por su tamano, se almacenaron en Google Drive y no se versionaron en el repositorio; este mantiene unicamente codigo, notebooks, configuraciones y etiquetas.

2.3 Estrategia adoptada

Del analisis anterior se derivó un plan de trabajo incremental, de menor a mayor dificultad, con la regla de no avanzar a una etapa sin haber validado la anterior:

1. Establecer un *baseline* de deteccion sobre imagenes limpias.
2. Descartar formalmente la oclusion con texturas fijas como prueba de concepto.
3. Implementar el ataque adversarial diferenciable sobre **un solo objeto**.
4. Evaluar la **transferencia** de esa textura a otros objetos y escenas.
5. Explorar una textura **universal** optimizada sobre multiples objetos.

3 Propuesta de Solucion

La solucion propuesta es un *pipeline* de ataque adversarial digital que optimiza una textura T contra YOLOv8-OBB. La textura se representa mediante un tensor libre θ y se obtiene aplicando la funcion sigmoide:

$$T = \sigma(\theta) \tag{1}$$

El uso de la sigmoide garantiza que los valores de la textura permanezcan en el rango $[0, 1]$ sin necesidad de recortes que interrumpan el flujo de gradiente. Durante todo el proceso, los pesos del modelo permanecen congelados: el unico elemento que se optimiza es la textura.

El procedimiento de cada iteracion de optimizacion es:

1. Obtener la textura $T = \sigma(\theta)$.
2. Reescalarla al tamano exacto del objeto y componerla sobre la imagen, usando unicamente operaciones diferenciables de *torch*.
3. Pasar la imagen resultante por el *forward* diferenciable de YOLO.
4. Calcular la perdida L_{det} sobre la confianza de la clase objetivo.
5. Retropropagar el gradiente hacia θ y actualizar la textura con Adam.

El proceso se repite hasta que el detector deja de reconocer el objeto. La textura resultante no se elige a mano: es descubierta por el gradiente.

4 Descripcion de la Solucion

4.1 Fase 1: Baseline con imagenes limpias

Se ejecuto YOLOv8-OBB sobre imagenes DOTA sin modificar, con el fin de verificar que el modelo detectara vehiculos correctamente antes de cualquier intervencion. Esta fase es indispensable: si el modelo no detectara el objeto en la imagen limpia, no podria atribuirse a la textura ningun efecto de evasion. El baseline produce un archivo `predictions.json` con las detecciones y sus poligonos, que se reutiliza en las fases posteriores.

4.2 Fase 2: Texturas fijas (prueba de concepto descartada)

Como punto de partida se probaron texturas fijas (tablero de ajedrez, ruido aleatorio, camuflaje verde/gris, franjas diagonales, gris solido), aplicadas sobre la region de cada objeto. Estas pruebas **no constituyen un ataque adversarial**, porque la textura no se

optimiza contra el modelo, y su efecto sobre la deteccion es esencialmente oclusion. No obstante, fueron utiles para validar la mecanica del *pipeline*: localizar objetos, aplicar una textura sobre su region, reejecutar YOLO y medir el cambio en las detecciones. Esta fase se documenta explicitamente como una etapa intermedia descartada, parte del proceso iterativo del proyecto.

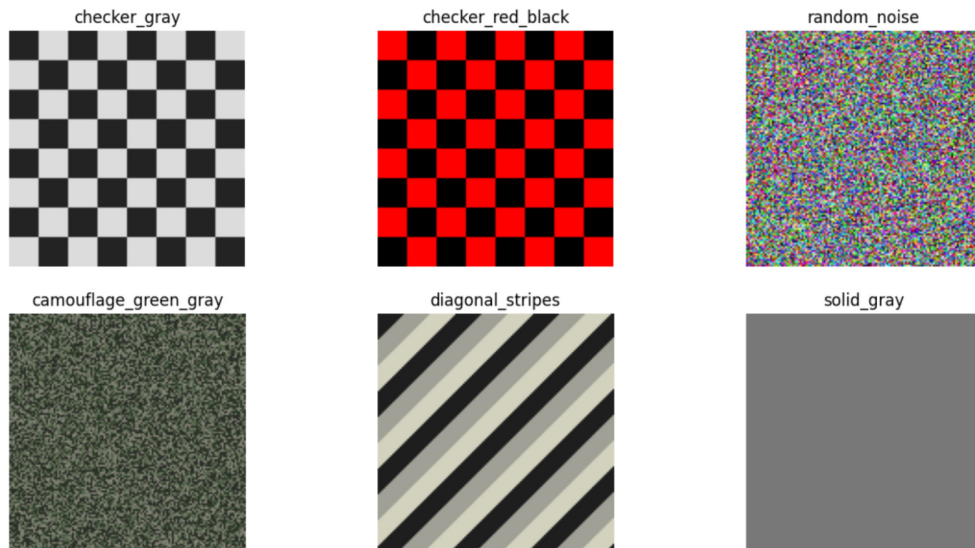


Figura 1: Galeria de seis texturas fijas probadas como prueba de concepto: checkerboard (gris y rojo-negro), ruido aleatorio, camuflaje, franjas diagonales y gris solido. Con cualquiera de ellas el detector pierde detecciones, pero el efecto es solo oclusión, no ataque adversarial.

El ranking por efectividad (caida de confianza promedio) muestra que las texturas mas visualmente contrastantes (checkerboard, camuflaje) logran mayor supresion, pero eso solo confirma que estan “tapando” el objeto:

4.3 Fase 3: Ataque adversarial diferenciable

4.3.1 Forward diferenciable de YOLOv8-OBB

Se verifico que la salida cruda del modelo tuviera el formato $[B, 4 + n_c + 1, A]$, donde B es el tamaño de *batch*, los primeros 4 canales corresponden a la caja, n_c a las clases, el ultimo al angulo, y A al numero de anclas. Con $n_c = 15$ en DOTA, el tensor observado fue:

```
raw.shape = [1, 20, 21504]
```

Un *sanity check* confirmo dos condiciones necesarias antes de entrenar: que las confianzas de clase se encuentran en el rango posterior a la sigmoide, y que el gradiente efectivamente llega a la textura (`theta.grad.abs().sum() > 0`).

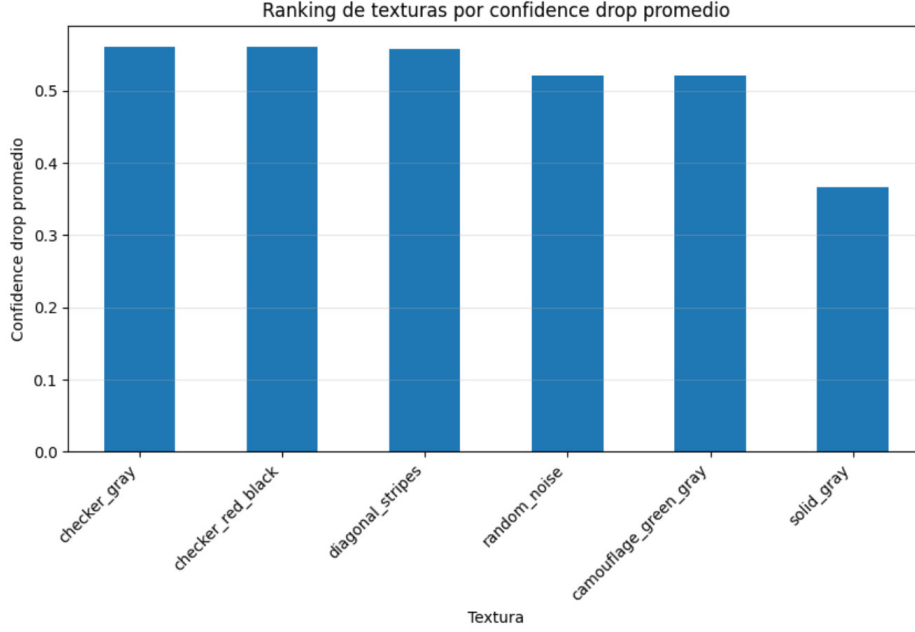


Figura 2: Ranking de texturas fijas por caida promedio de confianza. Aunque los números muestran efecto sobre la deteccion, el mecanismo es oclusion pura. Esto motivo pasar a un ataque verdaderamente adversarial.

4.3.2 Composicion diferenciable de la textura

La textura se reescala con `F.interpolate` al tamaño del objeto y se compone sobre la imagen mediante una mascara, con la operacion:

$$I_{adv} = I \odot (1 - M) + T_{colocada} \odot M \quad (2)$$

donde M es la mascara de la region del objeto. Es esencial que esta composicion use solo operaciones de *torch*: las funciones de PIL (*crop*, *paste*, *blend*, *resize*) rompen el grafo de gradientes y harian imposible el entrenamiento. La textura se aplica unicamente sobre la huella del objeto, sin expansion artificial de la region y sin mezcla por transparencia, de modo que el efecto medido provenga del patron y no de una oclusion ampliada.

4.3.3 Funcion de perdida

Una primera version de la perdida promediaba las confianzas dentro del objeto. Este enfoque resulto inadecuado: muchas anclas internas tienen confianza baja, por lo que el promedio diluye la senal de la deteccion dominante. La perdida corregida ataca la **maxima** confianza de clase objetivo entre las anclas cuyo centro cae dentro de la region del objeto:

$$L_{det} = \max_{a \in \mathcal{A}_{obj}} p(y \in \mathcal{C}_{target} \mid a) \quad (3)$$

donde $\mathcal{A}_{obj}$ es el conjunto de anclas dentro de la region y $\mathcal{C}_{target} = \{small\ vehicle, large\ vehicle\}$. Esta correccion fue determinante: el valor inicial de L_{det} paso a coincidir con la confianza reportada por `YOLO.predict()` (aproximadamente 0.83), lo que confirma que la perdida mide lo que el detector realmente percibe.

4.3.4 Configuracion de optimizacion

La primera prueba uso una configuracion deliberadamente agresiva, sin tecnicas de robustez, bajo el criterio de que si el ataque basico no funciona no tiene sentido entrenar versiones mas costosas:

- Optimizador: Adam, *learning rate* = 0.10.
- Iteraciones: 500.
- Sin *Expectation Over Transformation* (EoT) ni regularizacion.

4.4 Fase 4: Exploracion de textura universal

Para buscar una textura unica aplicable a multiples objetos se implemento un loop universal. La primera version, que optimizaba un objeto por iteracion, resulto inestable: la perdida oscilaba fuertemente porque la textura “perseguia” al ultimo objeto observado. La version corregida usa **entrenamiento por mini-batch con acumulacion de gradiente**: en cada iteracion se muestrean varios objetos de distintas escenas, se acumula el gradiente de todos antes de cada actualizacion, y se ejecuta un unico paso del optimizador por *batch*. Esta correccion estabilizo la convergencia, aunque, segun lo discutido en la Seccion 6, el caso universal resulta sustancialmente mas dificil que el ataque dirigido a un objeto.

5 Herramientas Aplicadas

5.1 Herramientas vistas en clase

El proyecto se apoya en conceptos de redes neuronales convolucionales y deteccion de objetos, asi como en el uso de PyTorch para definir tensores, calcular gradientes y ejecutar descenso de gradiente. La novedad respecto al uso habitual de estas herramientas es que el gradiente no se emplea para ajustar los pesos de un modelo, sino para optimizar la imagen de entrada (la textura), manteniendo el modelo congelado.

5.2 Herramientas no vistas en clase

Optimizacion adversarial sobre la entrada. A diferencia del entrenamiento estandar, donde se minimiza una perdida ajustando los pesos W del modelo, aqui se fijan los pesos y se optimiza la entrada. El objetivo es encontrar la textura T que minimiza L_{det} . Es el mismo mecanismo de *backpropagation*, pero el gradiente fluye hasta los pixeles en lugar de hasta los parametros.

Forward diferenciable de un detector. Para que el gradiente llegue a la textura, todas las operaciones entre la textura y la perdida deben ser diferenciables. Esto obliga a evitar la ruta de inferencia estandar de Ultralytics (que incluye NMS, no diferenciable) y a operar directamente con el `nn.Module` interno.

Expectation Over Transformation (EoT). Tecnica que dota de robustez a un ataque adversarial. En lugar de optimizar la textura sobre una unica vista, se aplican transformaciones aleatorias (rotacion, escala, brillo, ruido) en cada iteracion, de modo que el ataque funcione *en promedio* sobre un rango de condiciones. En este proyecto EoT se dejo preparado en el *pipeline* pero desactivado en las pruebas reportadas, por la decision metodologica de validar primero el ataque basico; su activacion gradual se plantea como trabajo futuro.

Regularizadores Total Variation (TV) y Non-Printability Score (NPS). TV penaliza diferencias bruscas entre pixeles vecinos, produciendo texturas mas suaves. NPS penaliza colores que una impresora no puede reproducir fielmente, acercando cada pixel a una paleta imprimible. Ambos son relevantes para una eventual version fisica del ataque y se incluyeron en el *pipeline* con peso cero en las pruebas digitales reportadas.

5.3 Uso de herramientas de IA generativa

Conforme a las instrucciones del proyecto, se documenta el uso de IA generativa como apoyo. La IA se empleo principalmente para acelerar la implementacion del *pipeline* y para depurar; las decisiones de diseno, la interpretacion de los resultados y la redaccion son propias. A continuacion se presentan los *prompts* mas representativos y la razon por la que funcionaron.

Prompt 1 — Diseno del ataque adversarial.

“Necesito implementar un ataque adversarial real contra YOLOv8-OBB. La textura debe ser un tensor de PyTorch optimizable con `requires_grad=True`, inicializado con

ruido, no un patron fijo. Necesito un loop de optimizacion donde la textura se aplique con operaciones diferenciables de torch/kornia (no PIL), se pase por el modulo interno de YOLO para obtener salidas crudas con gradiente, se calcule una perdida sobre la confianza de la clase objetivo, y se retropropague hacia la textura.”

Por que funciona: el *prompt* especifica de forma explicita las restricciones tecnicas que distinguen un ataque adversarial de una oclusion — tensor optimizable frente a patron fijo, operaciones diferenciables frente a PIL, modulo interno frente a `predict()`. Esa precision evita que la herramienta regenere una solucion de oclusion, que es la respuesta “por defecto” a una peticion vaga.

Prompt 2 — Correccion de la funcion de perdida.

“La perdida basada en el promedio de confianzas diluye la senal. Cambiala para que ataque la maxima confianza de la clase objetivo entre las anclas cuyo centro cae dentro de la region del objeto.”

Por que funciona: el *prompt* identifica el sintoma concreto (la senal se diluye) y prescribe el cambio especifico (maximo en lugar de promedio). Dirigir la optimizacion hacia la senal dominante alinea la perdida con lo que el detector reporta tras NMS.

Prompt 3 — Estabilizacion del loop universal.

“El loop universal entrena un objeto por iteracion y oscila. Cambialo a mini-batch con acumulacion de gradiente: muestrea varios objetos, divide cada perdida entre el tamano del batch, retropropaga para acumular gradiente, y ejecuta un solo paso del optimizador por batch.”

Por que funciona: la acumulacion sobre un mini-batch promedia la senal de varios objetos antes de cada actualizacion, lo que elimina la oscilacion causada por optimizar hacia un unico objeto a la vez.

6 Resultados

6.1 Ataque sobre un objeto

La prueba principal se realizo sobre un vehiculo de la clase *large vehicle*. Antes del ataque, YOLOv8-OBB detectaba tres cajas dentro de la region del objeto:

La optimizacion redujo la perdida L_{det} de manera estable y monotona, desde 0.829 en la primera iteracion hasta 0.0036 en la iteracion 500:

Cuadro 1: Detecciones dentro del objeto antes del ataque

Clase	Confianza	Estado
large vehicle	0.8176	Detectado
large vehicle	0.8046	Detectado
large vehicle	0.7864	Detectado

Cuadro 2: Evolucion de la perdida durante el ataque de un objeto

Iteracion	L_{det}
1	0.8285
25	0.2245
50	0.0772
100	0.0355
250	0.0088
500	0.0036

Tras aplicar la textura optimizada y reejecutar el detector real (con NMS), el resultado fue concluyente:

Detecciones dentro de la region objetivo

Original: 3

Atacado: 0

El detector continua operando normalmente sobre el resto de la imagen, pero deja de reconocer por completo el vehiculo intervenido. Esto confirma que el ataque adversarial funciona y que el efecto no es una oclusion generalizada.

6.2 Transferencia a otros objetos

Para medir el alcance del ataque, la textura optimizada sobre un unico objeto se evaluo sobre 20 objetos pertenecientes a tres escenas distintas. La metrica principal es la **tasa de exito del ataque** (ASR, *Attack Success Rate*), definida como la fraccion de objetos cuya deteccion se elimina por completo.

El resultado mas interesante no es la cifra global, sino su distribucion. La textura fue optimizada sobre un objeto de una escena de autobuses; transfiere de forma casi perfecta a otros objetos de escenas **visualmente similares** (P0002 y P0005), pero falla por completo en P0008, una escena de caracteristicas distintas. En los objetos de P0008 la caida de confianza fue marginal (entre 0.04 y 0.13 en tres de los cuatro casos).

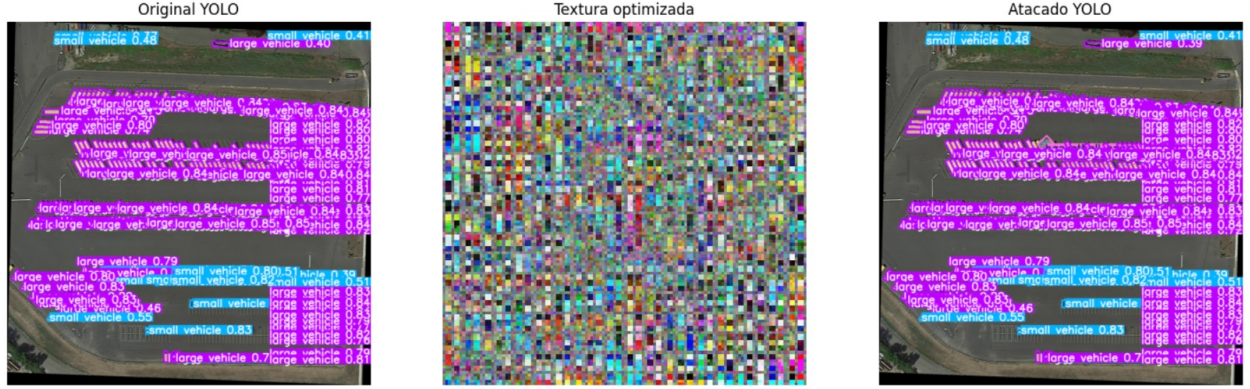


Figura 3: Resultado del ataque a un objeto. Izquierda: deteccion original en la imagen limpia (YOLO reporta multiples cajas de alta confianza de *large vehicle*). Centro: la textura adversarial optimizada por gradiente (patron colorido no natural). Derecha: tras aplicar la textura, YOLO deja de detectar el vehiculo dentro de esa region. El resto de la imagen sigue detectando otros objetos normalmente.

Cuadro 3: Transferencia de la textura por escena

Escena	Objetos	Exitos (det. $\rightarrow$ 0)	ASR
P0002.png	8	8	100.0 %
P0005.png	8	7	87.5 %
P0008.png	4	0	0.0 %
Total	20	15	75.0 %

Este comportamiento es un hallazgo en si mismo: el ataque **generaliza dentro de la distribucion de la escena de optimizacion, pero se degrada fuera de ella**. Es un resultado coherente con la literatura de *adversarial machine learning*, donde la transferibilidad de un ataque es limitada y depende de la similitud entre los dominios de optimizacion y de evaluacion.

6.3 Exploracion de textura universal

El loop universal con entrenamiento por mini-batch convergio de forma **estable** — a diferencia de la version de un objeto por iteracion, que oscilaba —, pero, en las iteraciones ejecutadas, alcanzo unicamente una **supresion parcial** de la confianza. Esto ilustra de forma concreta que obtener una textura unica eficaz sobre objetos y escenas heterogeneos es un problema considerablemente mas dificil que optimizar un ataque dirigido a un objeto. La obtencion de una textura universal plenamente eficaz se plantea como trabajo futuro.

6.4 Discussion

Los resultados cumplen el objetivo central del proyecto: demostrar que una textura optimizada por gradiente puede causar la evasión de YOLOv8-OBB en objetos aereos. Un punto metodologico relevante es que la comparacion correcta no es el total de detecciones de la imagen completa — en una escena DOTA hay decenas o cientos de vehiculos — sino las detecciones dentro de la region atacada. El ataque es exitoso localmente si elimina la deteccion del objeto objetivo, aunque la imagen global siga mostrando otras detecciones.

7 Limitaciones

- La mascara de aplicacion usa un *bounding box* rectangular en lugar del poligono OBB exacto, lo que introduce una pequena imprecision en los bordes del objeto.
- La textura optimizada es visualmente evidente; no se busco que pareciera un patron natural.
- La textura universal alcanzo unicamente supresion parcial en las iteraciones ejecutadas.
- No se evaluo la transferencia a modelos mas grandes (YOLOv8s-OBB, YOLOv8m-OBB) ni a otras arquitecturas de deteccion.
- No se realizo una prueba fisica impresa; toda la evaluacion es digital.

8 Trabajo Futuro

1. Completar la optimizacion de la textura universal con mas iteraciones y analizar su tasa de exito sobre multiples escenas.
2. Reintroducir EoT de forma gradual para dotar al ataque de robustez ante rotacion, escala e iluminacion.
3. Activar los regularizadores TV y NPS para obtener una textura imprimible y realizar una prueba fisica controlada.
4. Evaluar la transferencia del ataque a variantes mas grandes de YOLOv8-OBB.
5. Sustituir la mascara rectangular por la mascara del poligono OBB orientado.

9 Conclusion

El proyecto logro pasar de un *pipeline* inicial de texturas fijas — que solo producía oclusion — a un ataque adversarial real contra YOLOv8-OBB. Se valido el *forward* diferenciable del

modelo, se confirmo la existencia de gradiente hacia la textura y se diseño una funcion de perdida alineada con la confianza maxima del detector dentro del objeto.

El resultado principal es claro: un vehiculo inicialmente detectado con tres cajas de alta confianza dejo de ser detectado dentro de su region tras aplicar una textura optimizada por gradiente. La evaluacion de transferencia, ademas de cuantificar el alcance del ataque en un 75% de exito, revelo un comportamiento informativo — la textura generaliza dentro de la distribucion de la escena de optimizacion y se degrada fuera de ella —, que constituye un hallazgo coherente con la literatura del area.

El trabajo demuestra la viabilidad del enfoque, cuantifica honestamente sus alcances y sus limites, y deja una base reproducible y bien documentada para extender el ataque hacia texturas universales robustas y, eventualmente, pruebas fisicas.

Referencias

- [1] Xia, G.-S., Bai, X., Ding, J., Zhu, Z., Belongie, S., Luo, J., Datcu, M., Pelillo, M., & Zhang, L. (2018). *DOTA: A Large-Scale Dataset for Object Detection in Aerial Images*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- [2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. Disponible en: <https://github.com/ultralytics/ultralytics>
- [3] Thys, S., Van Ranst, W., & Goedeme, T. (2019). *Fooling Automated Surveillance Cameras: Adversarial Patches to Attack Person Detection*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW).
- [4] Brown, T. B., Mane, D., Roy, A., Abadi, M., & Gilmer, J. (2017). *Adversarial Patch*. Neural Information Processing Systems (NIPS) Workshops.
- [5] Athalye, A., Engstrom, L., Ilyas, A., & Kwok, K. (2018). *Synthesizing Robust Adversarial Examples*. Proceedings of the 35th International Conference on Machine Learning (ICML).
- [6] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). *Explaining and Harnessing Adversarial Examples*. International Conference on Learning Representations (ICLR).